Guion largo

Louvain y el paquete de datos

Detección de comunidades en la red de transferencias de Ethereum

Proyecto	Red de transferencias de Ethereum
Materia	Estructura de Datos — UIS
Responsable	Emerson Ballesteros
Tema	Tarea 3 — Louvain + paquete de datos
Entregable	Video de máximo 9 minutos (esta parte: 3 minutos)

Documento de estudio y guion de grabación

Contenido

0. Cómo usar este documento

1. El paquete de datos

2. Qué es Louvain

3. El código, función por función

4. Estructuras de datos usadas

5. Resultados del notebook

6. Validación

7. Limitaciones

8. Posibles mejoras

9. Guion de 3 minutos para grabar

10. Preguntas del jurado

11. Checklist antes de grabar

0. Cómo usar este documento

Este documento tiene dos partes:

1. **Guion de estudio:** explica el paquete de datos y Louvain con detalle, función por función.
2. **Guion de grabación:** al final está la versión de 3 minutos para leer frente a la cámara.

Primero entiende; después reduces.

---

1. El paquete de datos

1.1 Qué es el dataset original

El dataset se llama **Ethereum Exchanges (AlphaCore)**, de Stanford SNAP.

Dato	Valor
Transferencias	38.901.039
Tokens ERC-20	28
Bloques	6.525.201 a 10.000.000
Periodo	octubre 2018 a mayo 2020
Tamaño en CSV	8,3 GB

Cada fila es una transferencia:

```
token_address, from_address, to_address, value, transaction_hash, log_index, block_number
```

- `from_address`: quién envía.
- `to_address`: quién recibe.
- `value`: cantidad transferida, en la unidad mínima del token.
- `token_address`: contrato del token.
- `block_number`: bloque de Ethereum.

1.2 Por qué no se podía usar el CSV directamente

Tres problemas:

1. **GitHub** no permite archivos de más de 100 MB. El CSV de 8,3 GB no se puede subir.
2. **Colab** tiene alrededor de 12 GB de RAM. Cargar 38,9 millones de filas de texto es muy pesado.
3. **Velocidad:** leer un CSV gigante para cada consulta es lento.

No era solo un problema de espacio: también de tiempo de consulta y de memoria.

1.3 Qué se hizo

Se convirtió el CSV a **Parquet** usando **DuckDB**.

Parquet es un formato:

- **columnar:** guarda por columnas, no por filas;
- **comprimido:** usa ZSTD;
- **rápido de consultar:** se leen solo las columnas necesarias.

El comando fue:

```
python scripts/preparar_dataset.py convertir \
    --entrada data/raw/token_transfers_full.csv \
    --salida data/procesado/transferencias.parquet \
    --memoria 2GB --temporal data/procesado/tmp
```

Columnas que se conservaron

Columna	¿Por qué?
`from_address`	nodo origen
`to_address`	nodo destino
`value`	peso de la arista
`token_address`	permite filtrar por token
`block_number`	permite análisis temporal

Columnas que se quitaron

Columna	¿Por qué?
`transaction_hash`	identificador único; no se usa en los algoritmos
`log_index`	índice interno; no se usa

Resultado

Dato	Valor
Archivo	`transferencias.parquet`
Tamaño	807 MB
Filas	38.901.039
Tokens	28
Bloques	6.525.201 a 10.000.000

No se perdió ninguna fila. Solo cambió el formato y se quitaron dos columnas que no se usan.

1.4 Por qué quedó en 807 MB y no en 2,3 GB

El README hablaba de ~2,3 GB, pero el resultado fue mejor:

- no se guarda `transaction_hash`;
- no se guarda `log_index`;
- Parquet comprime muy bien las direcciones repetidas;
- ZSTD reduce mucho el tamaño.

El paquete es el **dataset completo**, no una muestra.

1.5 Cómo se verificó

Se hicieron consultas de control:

```
SELECT count(*) AS filas,
       count(DISTINCT token_address) AS tokens,
       min(block_number) AS bloque_min,
       max(block_number) AS bloque_max
FROM read_parquet('transferencias.parquet');
```

Resultado:

```
38.901.039 | 28 | 6.525.201 | 10.000.000
```

Eso confirma que el paquete está completo.

1.6 Para qué se usa el paquete

El paquete tiene dos usos:

1. **Estadísticas del dataset completo** con DuckDB:

- cuántas transferencias hay, - cuántos tokens, - rango de bloques, - actividad mensual, - top de tokens.

2. **Generar la muestra** que usan los algoritmos.

1.7 Por qué se genera una muestra

Los algoritmos están implementados en **Python puro**:

- **Bellman-Ford** es O(V · E). Con ~6 millones de nodos y 38,9 millones de aristas son ~2,3 × 10^14 operaciones. No termina en Colab.
- **Louvain** recalcula la modularidad muchas veces. Con millones de nodos tampoco termina.
- **Kruskal** necesitaría ordenar 38,9 millones de aristas y guardar millones de tuplas; riesgo de RAM.
- **PageRank** en Python puro con diccionarios también se vuelve lento.

La muestra no es para “hacer menos”; es para que los algoritmos **terminen**.

Cómo se genera la muestra

1. Se filtra **USDC** en el Parquet.
2. Se toma una muestra aleatoria reproducible con `semilla=42` y 65.000 filas.
3. Se descartan dirección cero, autolazos y valores ≤ 0.
4. Se conserva la **componente conexa más grande**.
5. Se normaliza `value` a USDC (6 decimales).

Resultado:

Dato	Valor
Nodos	1.371
Aristas	2.063
Token	USDC
Valores	normalizados a USDC

---

2. Qué es Louvain

2.1 La idea

Una **comunidad** es un grupo de nodos que interactúa mucho entre sí y poco con el resto.

Ejemplo:

```
Ana   <--> Beto     (mucho)
Carla <--> Dani     (mucho)
Beto  -->  Carla    (poquito)
```

Hay dos grupos naturales:

- `{Ana, Beto}`
- `{Carla, Dani}`

Louvain intenta encontrar esos grupos **sin que le digamos cuántos hay**.

2.2 La modularidad Q

Q mide si una partición es mejor que el azar.

Fórmula por comunidad:

```
Q = suma_c [ L_c / m - (K_c / (2m))^2 ]
```

Donde:

- `L_c`: peso de las aristas internas de la comunidad `c`.

- `K_c`: suma de grados de los nodos de `c`.
- `m`: peso total del grafo.

Interpretación:

- `Q > 0,3`: hay estructura de comunidades.
- `Q = 0`: la partición no es mejor que el azar.
- `Q` cercano a 1: comunidades muy marcadas.

2.3 El algoritmo

1. Cada nodo empieza en su propia comunidad.
2. Se recorre la lista de nodos en orden aleatorio.
3. Para cada nodo, se prueba moverlo a la comunidad de cada vecino.
4. Se acepta el movimiento solo si Q aumenta.
5. Cuando una pasada completa no cambia nada, termina.

Es una **heurística**: no garantiza el óptimo global, pero funciona bien y es rápida en la práctica.

2.4 Por qué se dice “fase principal”

Louvain completo tiene dos fases:

1. **Fase local:** mover nodos entre comunidades.
2. **Fase de agregación:** convertir cada comunidad en un super-nodo y repetir.

En el proyecto implementamos la **fase 1**. Es la parte principal y es fácil de explicar. La fase 2 queda como mejora.

---

3. El código, función por función

El código está en `src/comunidades.py` y también va dentro del notebook.

3.1 `simetrizar`

Qué hace: convierte el grafo dirigido en no dirigido.

Louvain trabaja con comunidades, no con direcciones. Si existe `A → B` y `B → A`, se suman los pesos.

```
pesos_por_par = defaultdict(float)

for origen, vecinos in adj.items():
    for destino, peso in vecinos:
        a, b = sorted((origen, destino))
        pesos_por_par[(a, b)] += peso
```

Línea importante:

```
a, b = sorted((origen, destino))
```

Sirve para que `(A, B)` y `(B, A)` sean el mismo par y no se cuenten dos veces.

Después:

```
adj_no_dirigida[a].append((b, peso))
adj_no_dirigida[b].append((a, peso))
```

La arista queda en los dos sentidos.

3.2 `calcular_grados`

Qué hace: suma los pesos de las aristas de cada nodo.

```
for nodo, vecinos in adj_no_dirigida.items():
    suma = 0.0
    for _, peso in vecinos:
        suma += peso
    grados[nodo] = suma
```

Ese es el grado ponderado `k_i`.

3.3 `modularidad`

Qué hace: calcula Q.

Paso 1: grados y m

```
grados = calcular_grados(adj_no_dirigida)
suma_de_grados = sum(grados.values())
peso_total = suma_de_grados / 2.0
```

Se divide entre 2 porque cada arista suma su peso a los dos extremos.

Paso 2: K_c

```
grado_por_comunidad[comunidad] += grado
```

Es la suma de grados de los nodos de cada comunidad.

Paso 3: L_c

```
if comunidades[nodo] != comunidades[vecino]:
    continue
if nodo < vecino:
    interno_por_comunidad[comunidades[nodo]] += peso
elif nodo == vecino:
    interno_por_comunidad[comunidades[nodo]] += peso / 2.0
```

Línea importante:

```
if nodo < vecino:
```

Hace que cada arista no dirigida se cuente **una sola vez**.

Paso 4: fórmula final

```
q += interno / peso_total - (grado_total / (2.0 * peso_total)) ** 2
```

Esa es exactamente la fórmula de Q por comunidad.

3.4 `louvain`

Qué hace: mueve nodos para maximizar Q.

Generador reproducible

```
generador = random.Random(semilla)
```

No usa el generador global. Así el resultado es reproducible y no afecta a otros módulos.

Comunidades iniciales

```
for identificador, nodo in enumerate(nodos):
    comunidades[nodo] = identificador
```

Cada nodo empieza solo.

Pasadas

```
for numero_pasada in range(max_pasadas):
    hubo_cambio = False
    generador.shuffle(nodos)
```

El orden aleatorio evita favorecer siempre a los mismos nodos.

Probar comunidades vecinas

```
comunidades_vecinas = set()
for vecino, _ in adj_no_dirigida.get(nodo, []):
    comunidades_vecinas.add(comunidades[vecino])
```

Se usa un `set` para no probar dos veces la misma comunidad.

```
for comunidad_vecina in sorted(comunidades_vecinas):
```

Se ordena para que el recorrido sea reproducible.

Probar el movimiento

```
comunidades[nodo] = comunidad_vecina
q_con_movimiento = modularidad(adj_no_dirigida, comunidades)
if q_con_movimiento > mejor_q:
    mejor_q = q_con_movimiento
    mejor_comunidad = comunidad_vecina
comunidades[nodo] = comunidad_actual
```

- Se mueve temporalmente.
- Se calcula Q.
- Si mejora, se guarda.
- Se deshace para probar el siguiente.

Aceptar y terminar

```
if mejor_comunidad != comunidad_actual:
    comunidades[nodo] = mejor_comunidad
    hubo_cambio = True
```

```
if not hubo_cambio:
    break
```

Si nadie se movió, termina.

3.5 `resumen_comunidades`

Qué hace: agrupa los nodos por comunidad y devuelve nombres legibles.

```
grupos[comunidad].append(nombre)
```

Si hay etiquetas de exchanges, usa el nombre; si no, usa la dirección.

3.6 `renumerar_comunidades`

Qué hace: cambia los IDs por 0, 1, 2, ...

Louvain devuelve IDs como 300 o 692. Para leer mejor:

```
if id_viejo not in mapa_de_ids:
    mapa_de_ids[id_viejo] = len(mapa_de_ids)
comunidades_nuevas[nodo] = mapa_de_ids[id_viejo]
```

No cambia la partición; solo el nombre.

3.7 `desde_aristas`

Qué hace: construye el grafo no dirigido desde `grafo.aristas`.

```
adj_dirigida[origen].append((destino, float(peso)))
return simetrizar(adj_dirigida)
```

Es el punto de contacto entre la carga de datos y Louvain.

3.8 `louvain_desde_aristas`

Qué hace: atajo para el notebook.

```
adj_no_dirigida = desde_aristas(aristas)
return louvain(adj_no_dirigida, semilla=semilla, max_pasadas=max_pasadas)
```

---

4. Estructuras de datos usadas

Estructura	Dónde	Para qué
Lista de adyacencia (`dict` de listas)	`simetrizar`, `louvain`	recorrer vecinos
Diccionario	`comunidades`, `grados`	guardar comunidad y grado
Conjunto (`set`)	`comunidades_vecinas`	no repetir comunidades
`defaultdict`	`simetrizar`, `modularidad`	acumular pesos
`random.Random`	`louvain`	reproducibilidad

No usamos matrices de adyacencia porque serían enormes y llenas de ceros. La lista de adyacencia guarda solo las conexiones que existen.

Complejidad

- `simetrizar`: O(E).
- `calcular_grados`: O(V + E).
- `modularidad`: O(V + E).
- `louvain`: O(pasadas · V · grado · (V + E)) en esta versión sencilla.
- `renumerar_comunidades`: O(V).

La versión de producción de Louvain usa ΔQ incremental y agregación, con complejidad aproximada O(V log V).

---

5. Resultados del notebook

Sobre la muestra de USDC:

```
Grafo dirigido:
  nodos   : 1.371
  aristas : 2.063

Grafo no dirigido:
  aristas : 1.482

Modularidad Q = 0.735
Número de comunidades = 70
```

Las comunidades más grandes:

- 404 nodos.
- 98 nodos.
- 95 nodos.
- 86 nodos.
- 80 nodos.

Estabilidad con semillas

Semilla	Q	Comunidades
1	0.731	71
7	0.736	69
42	0.735	70
123	0.732	70
2024	0.730	69

Conclusión: Q se mantiene entre 0,730 y 0,736; el resultado es estable.

---

6. Validación

¿Cómo sabemos que el resultado no es casualidad?

1. **Varias semillas:** Q cambia muy poco.
2. **Comparación con otras alternativas:** Label Propagation y `python-louvain`.
3. **Revisión de comunidades:** tamaños y coherencia.
4. **Comparación con etiquetas de exchanges:** si se cargan las etiquetas, Binance 1, Binance 2 y Binance 3 caen en la misma comunidad.

La validación no demuestra que sea la mejor partición posible, pero sí que el resultado es estable y razonable.

---

7. Limitaciones

1. **Solo fase principal:** no incluye agregación.
2. **Recalcula Q completo:** es fácil de leer, pero lento.
3. **Heurístico:** no garantiza el óptimo global.
4. **Límite de resolución:** comunidades muy pequeñas pueden pasar desapercibidas.
5. **Muestra:** no es el dataset completo; es una muestra conectada de USDC.
6. **Sin etiquetas en la versión básica:** las salidas muestran direcciones.

---

8. Posibles mejoras

1. **ΔQ incremental:** no recalcular Q completo.
2. **Fase de agregación:** Louvain completo.
3. **Parámetro de resolución γ.**
4. **Varias semillas y elegir la mejor Q.**
5. **Comparar con `python-louvain` y Label Propagation.**
6. **Usar arreglos/CSR** para grafos de millones de aristas.

---

9. Guion de 3 minutos para grabar

0:00–0:20 — Apertura

> “Hola, soy Emerson Ballesteros. Presento la Tarea 3, detección de comunidades con Louvain, y explico cómo pasamos del dataset original al paquete optimizado.”

0:20–0:55 — El paquete

> “El dataset original tiene 38.901.039 transferencias y 28 tokens, y en CSV pesa 8,3 GB. No cabe cómodamente en Colab ni se puede subir a GitHub. Lo convertimos a Parquet con DuckDB: conservamos origen, destino, valor, token y bloque; quitamos hash y log_index. Quedó en 807 MB, con las mismas 38,9 millones de filas. No es una muestra: es el dataset completo en un formato más eficiente.”

> “El paquete se usa para estadísticas completas con DuckDB. Para los algoritmos generamos una muestra de USDC, porque Bellman-Ford y Louvain en Python puro no pueden con 38,9 millones.”

0:55–1:30 — Louvain

> “Una comunidad es un grupo de direcciones que interactúa mucho entre sí y poco con el resto. Louvain maximiza la modularidad Q, que compara las conexiones internas con las esperadas por azar. L_c es el peso interno, K_c la suma de grados y m el peso total. El algoritmo empieza con cada nodo en su comunidad, prueba moverlo a la comunidad de sus vecinos, acepta solo si Q sube y termina cuando nada cambia.”

1:30–2:05 — Código

> “El código está en comunidades.py y usa nombres en español. Simetrizar convierte el grafo dirigido en no dirigido. Calcular_grados suma pesos. Modularidad calcula Q. Louvain hace las pasadas con random.Random para que sea reproducible. Renumerar_comunidades cambia los IDs a 0, 1, 2 para leer mejor. Usamos lista de adyacencia, diccionarios y conjuntos.”

2:05–2:35 — Resultados

> “Sobre la muestra de USDC, con 1.371 nodos y 2.063 aristas, Louvain encuentra 70 comunidades con Q = 0,735. La comunidad más grande tiene 404 nodos. Eso indica una estructura clara.”

2:35–2:50 — Validación

> “Probamos cinco semillas: Q se mantiene entre 0,730 y 0,736, y las comunidades entre 69 y 71. El resultado es estable.”

2:50–3:00 — Cierre

> “La limitación es que implementamos la fase principal, que recalcula Q y no escala a 38,9 millones. Por eso el dataset completo se analiza con DuckDB y los algoritmos se demuestran sobre la muestra. Gracias.”

---

10. Preguntas del jurado

¿Por qué no Louvain sobre 38,9 M?

> “Porque es Python puro y recalcula modularidad; con millones de nodos no termina. El dataset completo se usa para estadísticas.”

¿Qué estructura usa Louvain?

> “Lista de adyacencia con diccionarios, diccionario de comunidades y conjuntos.”

¿Qué significa Q = 0,735?

> “Que hay más conexiones internas de las esperadas por azar. Q > 0,3 ya indica estructura; 0,735 es alto.”

¿Por qué 70 comunidades?

> “No lo fijamos; Louvain decide. Controlamos semilla y máximo de pasadas.”

¿Por qué el paquete pesa 807 MB y no 2,3 GB?

> “Quitamos transaction_hash y log_index, y Parquet con ZSTD comprime las direcciones repetidas.”

¿Por qué no suben el CSV?

> “Pesa 8,3 GB y GitHub limita a 100 MB. El Parquet va a Drive/Colab.”

---

11. Checklist antes de grabar

- ☐ Notebook abierto en la teoría de Louvain.
- ☐ Celda de carga ejecutada.
- ☐ Salida `Q = 0.735` visible.
- ☐ Tabla de semillas a mano.
- ☐ Tabla del paquete a mano.
- ☐ Cronómetro en 3:00.
- ☐ No mostrar celdas que fallen.
- ☐ Decir “muestra” cuando hables de Louvain y “dataset completo” cuando hables de DuckDB.